Conceptual Architecture: Gemini CLI

CISC 322 Assignment 1

Authors:

Connor Zakary – 22ksh3@queensu.ca

Ethan Whitcher – ethan.whitcher@queensu.ca

Devan Pearson – 22kgx1@queensu.ca

Liam Mathers - 22rbv1@queensu.ca

Marcela Rojas - 22mr6@queensu.ca

Abstract

Gemini CLI is an open-source command line interface for Google's Gemini AI model, which can be used by developers directly in their terminals. It allows users to directly prompt Large Language Models (LLMs) within their IDEs. Gemini CLI is a tool designed to bridge the gap between a user's terminal/IDE and cloud-based LLMs. It integrates LLM assistance within command lines, focusing on automation to make writing code with the integration of LLMs more efficient. Gemini CLI can read and edit files, perform shell commands to assist in programming.

This report delivers an architectural analysis of Gemini CLI's major components and examines its position in software development. It focuses on the strengths of Gemini CLI, like external API communication and orchestration. While also acknowledging issues with the system, such as reliance on external APIs and possible unpredictability. Use cases of Gemini CLI are also discussed, as well as its interactions with outside interfaces. As well as an overview of the architectural style of Gemini CLI.

Table of Contents

1 Introduction ---------- 4

2 Architectural Overview ---------- 4

a. Derivation Process ---------- 4

b. High-level Architecture Overview ---------- 5

i. Gemini CLI Overall Structure ---------- 5

ii. CLI and User Interface ---------- 5

iii. Core Orchestrator ---------- 5

iv. Tool Registry ---------- 5

v. Gemini API ---------- 6

c. Architectural Styles ---------- 6

i. Layered Architecture ---------- 6

ii. Interpreter Architecture ---------- 7

iii. Alternative Architectural Styles Considered ---------- 8

3 External Interfaces ---------- 8

4 Use Cases ---------- 9

a. Use Case 1: Batch file editing – Photo Example ---------- 9

Use Case 2: Web App Dashboard for Business Intelligence ---------- 10

6 Lessons Learned ---------- 11

5 Conclusion ---------- 12

7 Data Dictionary ---------- 13

8 Naming Conventions ---------- 13

9 AI Collaboration Report ---------- 13

a. Task Assignment and Division of Responsibilities ---------- 13

b. AI Interaction Protocol and Prompting Strategy ---------- 14

c. Reflection on Human-AI Team Dynamics ---------- 14

d. Quantitative Impact ---------- 14

e. Overall Assessment ---------- 15

1 Introduction

Large Language Models (LLMs) are undeniably a growing part of modern software development, and society at large. The ability to interpret high-level prompts and produce quality code, analysis, and automation to solve problems presented by users is highly applicable to programming cases. This changes the way that code is written by professionals worldwide. The next logical step is direct integration of LLMs into the programming process. One such integration is the direct integration of LLMs into CLIs. Web based tools can offer very helpful insight, but CLI based tools ignore the hurdles of their web-based counterparts, like workflow interruption. Here, we see the desire for systems like Gemini CLI.

Gemini CLI, developed by Google, is an open-source command-line interface that integrates Gemini large language models directly into terminal-based workflows. Where most CLIs execute fixed commands, Gemini CLI considers the intentions of users to create plans, utilize local tools, file access, as well as cloud-based AI services to assist in coding. Gemini CLI is defined by an interpreter style architecture, in which user intent is incrementally evaluated and translated into executable actions. This approach is combined with a layered structure that separates user interaction, orchestration, tool execution, and external API communication, enabling extensibility while maintaining clear boundaries between system responsibilities.

This report aims to give a comprehensive understanding of Gemini CLI's conceptual architecture by giving an overview of the architectural style. As well as examining external interfaces which Gemini CLI interacts with, discussing use cases of Gemini CLI, and discussing takeaways that can be learnt from Gemini CLI's architecture. The architectural overview section explains the process which led us to decide that Gemini CLI uses an interpreter style architecture as well as layered style. It then gives a high level overview of the structure, user interface, Core Orchestrator, Tool Registry, and Gemini API. The external interface section discusses the different interfaces which Gemini CLI interacts with throughout use. The use case section details two use cases. First, the mass editing of all files in a directory. Explaining how Gemini CLI interacts with local files to convert all photos in a directory into .png files. Second, how Gemini CLI would work to design a Web App Dashboard for Business Intelligence.

We conclude with a summary of our findings, as well as the lessons we learned from our investigation into the architectural style of Gemini CLI. These final sections discuss the features and components of Gemini CLI and why they align with the architectural style that we elected to label Gemini CLI. Also attached are sections discussing the terminology used in the report to avoid any confusion. Finally, we attached an AI collaboration report. This discusses any use of artificial intelligence in the creation of this report and how we worked with AI to understand the structural architecture of Gemini CLI.

2 Architectural Overview

a. Derivation Process

Deriving the architecture of the Gemini CLI began with reviewing relevant documentation coming from sources directly tied to the software tool. The official Gemini CLI architecture documentation and GEMINI.md file of the open-source repository served as a great foundation for understanding the overall orchestration of this tool [1][2]. The code is organized as a monorepo, that is common amongst AI agents which typically require tight coordination between

components. Further reading into the Core package documentation and Tools API reference provided a deeper understanding on the systems distribution of responsibilities, such as how the agentic loop tool interacts with the Gemini API. The model dynamically deciding the next action based on accumulated results strongly indicated an interpreter style design which helped understand patterns within the overall system architecture [3].

After building a strong understanding through documentation and supplementary architecture analysis articles, the major functional areas such as the user interaction, orchestration, tool execution, and API communication were identified. This information was then used to formulate use cases and sequence diagrams to trace the control and data flow between functional areas as seen in section 4. Between the referenced documentation and components repeatedly used across use cases, these were then translated to subsystems in the final box-and-arrow diagrams.

b. High-level Architecture Overview

i. Gemini CLI Overall Structure

As aforementioned, Gemini CLI is a monorepo that is cleanly divided between its two main packages, the CLI package and the Core package. The CLI package serves as the frontend and is responsible for everything the user sees and interacts with. All processing is then handed over to the backend Core package, which is responsible for orchestrating the agentic loop, constructing prompts, managing tool executing, and communicating with the external Gemini API.

ii. CLI and User Interface

The CLI package is the user-facing terminal that is built upon React and Ink (React library for rendering components to CLI). Responsibilities of the package include parsing user input, rendering streamed responses, displaying tool confirmation prompts, and managing UI customization. This is the only layer that the user directly interacts with while all other components sit behind it, strongly suggesting a layered architecture. The component depends on the Core Orchestrator and receives both final outputs and confirmation requests before processing is executed.

iii. Core Orchestrator

The core orchestrator is the central coordinator of the entire Gemini CLI which runs the agentic loop and drives all processing. Upon receiving user input, the orchestrator will construct a request for the Gemini API, transfer it via HTTPS, interpret the response, and either return a final answer or routes to Tool Registry. If the response is a final answer, this will be sent to the CLI for display to the user. If a tool call is needed, the orchestrator would route to the Tool registry for execution of that tool and relaying the result back to the API for further processing. An example of these tools is the ReadFileTool, which takes an absolute file path and returns its contents. This cycle is managed internally through a loop that iterates between sending requests to the API and executing tool calls until a final response can be returned to the frontend.

iv. Tool Registry

The Tool Registry acts as a repository for all available tools that can be called upon by the core. It is responsible for registering tools, storing their schemas, and exposing them to the core so it knows what can be requested. After requesting a tool, core will manage the entire execution lifecycle including validating parameters, checking for user confirmation when required, and execution. The actual tool implementations are contained in their own respective tools file within the core folder.

v. Gemini API

The Gemini API is an external service that acts as the reasoning engine for the entire system. All reasoning and decision making is done through the API, meaning the agentic loop makes at least one HTTPS call to this service each iteration. If it returns a final answer, the text will be passed to the CLI for display. If a tool is requested, Core will consult the registry and execute the tool, returning the result back to the API as additional context. Due to not having its own reasoning capabilities, all reasoning depends on this external service which can introduce constraints around network availability and rate limiting. These constraints shape architectural decisions, such as automatically switching to a lower tier model when hitting rate limits.

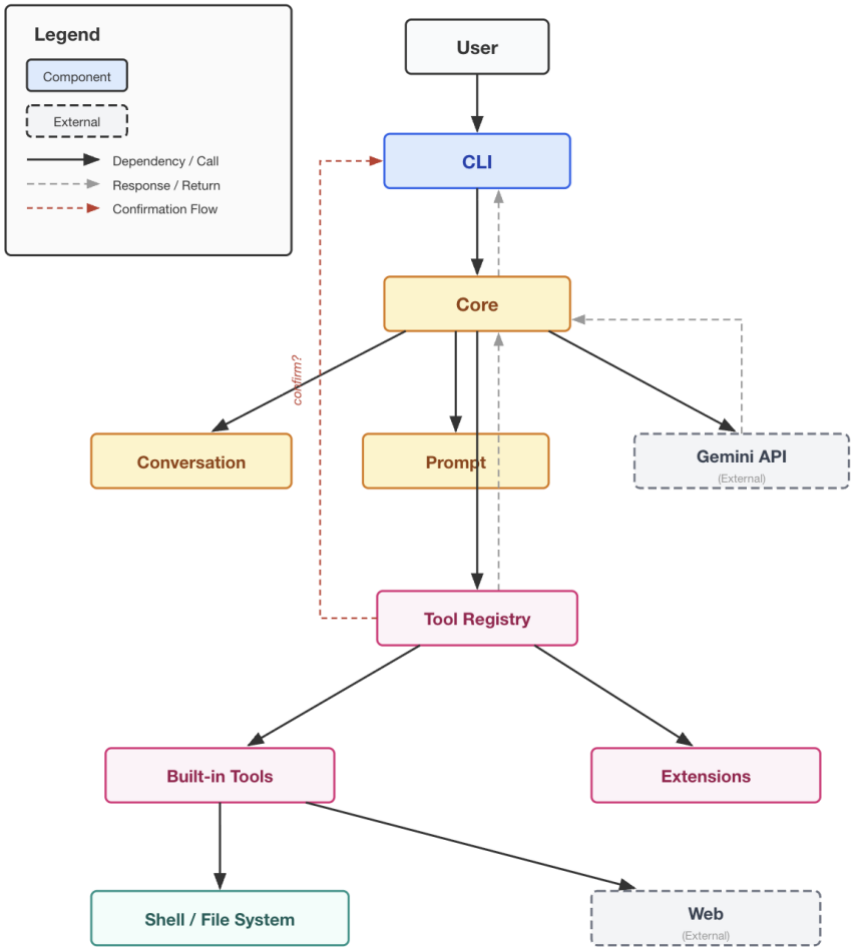

Figure 1: Box and Arrow Diagram for High-Level Gemini CLI Architecture

c. Architectural Styles

i. Layered Architecture

Gemini CLIs overall architecture is organized as a layered software architecture where each layer provides services to the one above it. The topmost layer is the CLI package which handles input, displays, confirmation prompts, and is the only client-facing layer. Below it sits the core package layer, which has been previously covered as the core orchestrator behind the entire system. The core will iterate through the agentic loop, which depends on the Gemini API and

Tool services below. The tools then interact with the lowest level of architecture, by either operating directly on the users file system or reaching out to external web sources.

This layered organization supports modularity within the codebase, as each layer can be developed or replaced independently. As highlighted in the Gemini CLI architecture docs, modularity is a key design principle which allows for future extensions, such as replacing the CLI for a different frontend interface. A major benefit of layered architecture is having a clear separation of concerns where each layer has designated responsibilities. This reaps many benefits including but not limited to easier to maintain code, independent testing, and extensibility at specific layers. An important caveat to this layered architecture is that there are specific exceptions to this rule, especially when it comes to security and file permissions. When a potentially destructive tool is requested, this request will bypass the core layer and go straight to the user at the CLI layer for confirmation. While this breaks the nature of layered architectures where communication only flows downwards through adjacent layers, the action is necessary to ensure safety of the user's system before proceeding.

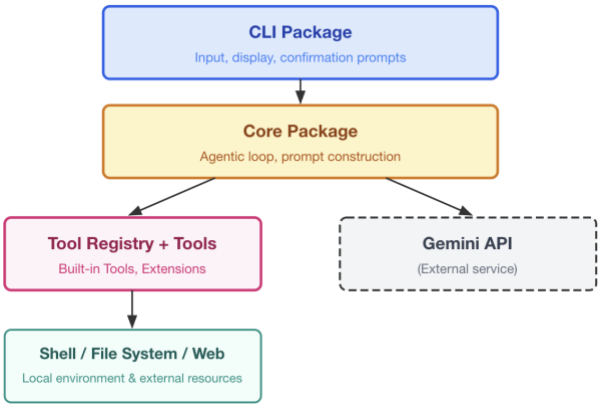

Figure 2: Layered Architecture of Gemini CLI

ii. Interpreter Architecture

While a layered architecture helps describe the overall layout of Gemini CLI, an interpreter architecture characterizes the runtime behavior of the Core's agentic loop. This pattern fits the system because Gemini CLIs agentic loop doesn't know in advance what its next request will be. Program execution proceeds dynamically, where the next steps always depend on the previous state.

As outlined in Figure 3, the architecture can be mapped to a specific element found within interpreter architecture. The users prompt and tool schemas form the program being interpreted, which serves as the task and available tools within memory. As conversation between the user and CLI happens, the accumulated conversation history and tool results become the current state of the program. The execution engine of this system is a combination of the Gemini API and Core orchestration logic. The API is responsible for all logic and reasoning, telling the Core what should be done. The Core then executes this, whether that be return a response to the user or lookup another tool. Lastly, the state of the execution engine is handled by the agentic loops current phase which consists of waiting for an API response or executing tool calls.

From the design perspective of Gemini CLI, an interpreter style architecture is excellent for providing the necessary flexibility of handling arbitrary multi-step requests without a hardcoded logic workflow. The sequence of API calls and tool requests cannot be predetermined which is why it makes sense to dynamically handle the outputs of such. This architecture also allows for extensibility by separating the Core's agentic loop engine from other components. New tools can be developed and added to the registry without requiring any change to the execution loop. There are also notable tradeoffs that must be considered, especially because the execution engine is a non-deterministic AI model. While general reasoning remains consistent, giving Gemini the same prompt won't produce an identical result. This could lead towards inefficiencies and unbounded execution, where models will make multiple tool calls rather than converge on a final answer.

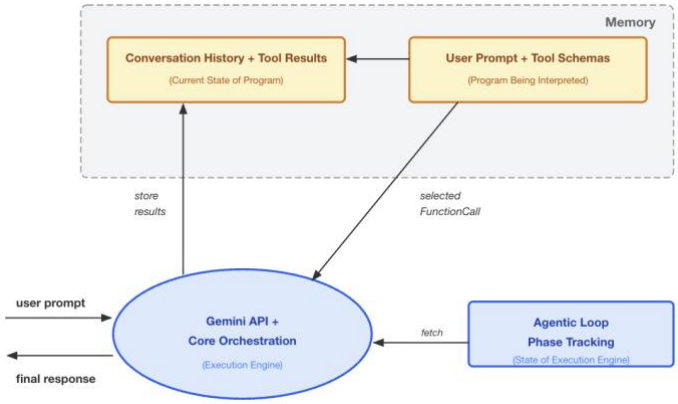

Figure 3: Gemini CLI Interpreter Architecture

iii. Alternative Architectural Styles Considered

During this derivation process, multiple architectural styles were considered when characterizing the overall makeup of the system. While some did not fit as well as the interpreter style or layered architecture style, they still possess notable similarities. An easy characterization the Gemini CLI falls under is a textbook client-server style where the CLI is the client and server is the Gemini API. While correct, this fails to capture the fundamental innerworkings of the system which include the agentic loop, layered separation between components, and tool execution. Characterizing the Gemini CLI as a client the requests something from the server and gets a response would be a vast oversimplification of the systems overall makeup.

3 External Interfaces

The primary external interface of Gemini CLI is the command line UI. Gemini CLI is unique among modern AI applications in that it sits natively on your computer. Gemini CLI is a relatively lightweight model that operates within the scope of the device it's installed on. CLI, standing for Command-Line Interface, encapsulates exactly how one interacts with the model.

The secondary interface Gemini CLI uses to interact with its environment is the file system. Gemini CLI can ingest photos, text files, code, and more as input context, and can then be used to produce documentation, rehashed code, or analysis. This bi-directional I/O makes

Gemini CLI suitable for a wide variety of tasks, from software engineering to data processing, to personal file management.

Externally, Gemini CLI operates as a client to remote AI services over network APIs, typically using HTTPS. In enterprise contexts, this traffic may traverse proxies or controlled network boundaries. Additional extensibility is provided through plugin or tool interfaces, allowing the model to invoke shell commands, linters, or testing frameworks. Finally, logging and telemetry interfaces expose diagnostics, usage data, and debug traces for observability.

This collection of interfaces position Gemini CLI as a layered, interpreter system, connecting human input, local resources, and cloud hosted AI services.

4 Use Cases

Gemini CLI supports a wide range of uses cases across both personal and enterprise use cases. A common intended application is for it to be used to perform batch operations on groups of files. For instance, it can take a directory, convert all images in it to PNG, then rename them according to EXIF data. An alternative might be for a small business owner to use it to develop a web application dashboard for business intelligence. The following sequence diagrams will examine both workflows and how the AI agent interacts with local files.

a. Use Case 1: Batch file editing – Photo Example

This example use case involves prompting Gemini CLI to change all photos in a given directory to PNG and rename them based on the date in metadata. The design-time sequence for this use case begins with the **startup and scoping phase**, where the user opens a terminal and launches Gemini CLI from within (or pointed at) the target image directory. Establishing this working directory is functionally equivalent to selecting a "project" context: it constrains Gemini CLI's file discovery and ensures any subsequent operations (reads, writes, renames) remain localized to the intended dataset.

Once Gemini CLI is running, the user provides a high-level batch-editing request, such as: "Convert all images in this directory to JPEG (quality 90), then rename them using EXIF "DateTimeOriginal"; if EXIF is missing, fall back to file modified time; avoid overwrites and show a dry run first." At this point, Gemini CLI acts as the orchestration layer, translating an intent-level specification into an executable plan. Next, rather than proposing a GUI set of objects, Gemini CLI would propose a set of shell actions, file enumeration, EXIF data extraction, conversion commands, and naming rules, to be then used as a deterministic pipeline for execution.

The next step is for Gemini to enumerate photos in the directory, looking for JPG, JPEG, etc., and retrieve metadata using an external tool like "exiftool", to pull timestamps from the field "DateTimeOriginal". This external utility boundary is a key interface: Gemini CLI does not "own" EXIF parsing or image transcoding internally, but instead coordinates specialized tools and normalizes their outputs into a consistent naming scheme. When EXIF data is unavailable or inconsistent, Gemini CLI falls back to filesystem metadata (mtime), ensuring the batch process remains total (i.e., defined for every file) rather than failing on edge cases.

After a brief dry run to ensure there are no collisions in new file names, the workflow enters the execution stage, Where Gemini CLI issues a repeated conversion-and-rename sequence for each file. Image conversion is delegated to a dedicated processor (commonly ImageMagick), while filesystem operations (creating an output directory, writing converted files, renaming/moving outputs) are handled by the OS through standard file APIs. If metadata preservation is required, Gemini CLI extends the pipeline with a post-processing pass that copies EXIF tags from the original file to the converted output (again using exiftool). The process concludes with a summary report: number of files processed, any skipped items, collisions resolved, and errors encountered.

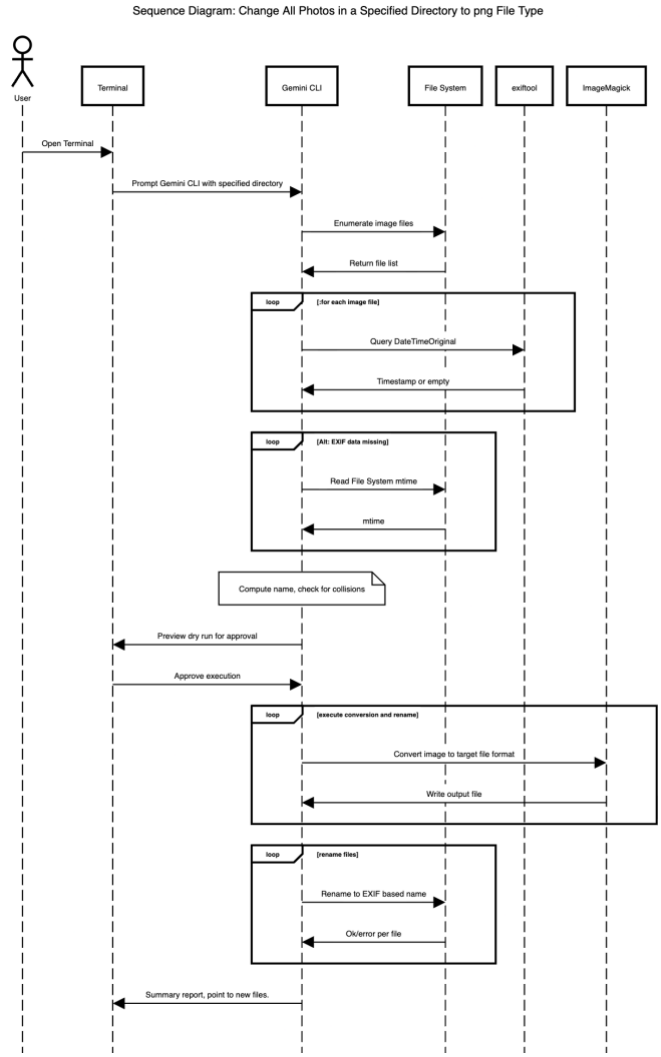

Figure 4: Sequence Diagram of Batch Photo Processing Request Use Case

Use Case 2: Web App Dashboard for Business Intelligence

The design-time sequence for this use case begins with the **startup and access phase**, where a small business user (often an owner-operator or manager) opens the BI dashboard in a web browser and authenticates. The application's frontend (typically a single-page web app) loads the

dashboard shell; navigation, filters, and empty widget containers, while the back end establishes the user's workspace context (company, locations, permissions, and enabled data sources). Because small businesses often lack dedicated IT staff, the system prioritizes a low-friction onboarding flow: clear role-based access, guided setup, and safe defaults that produce value quickly without complex configuration.

Once the session is initialized, the dashboard enters the data connectivity and ingestion phase. The user either connects common small-business systems (e.g., POS, e-commerce storefront, payment processor, accounting package, marketing platform) or relies on periodic file imports (CSV exports) if integrations are unavailable. The application's Integration Layer handles OAuth/API credentials, schedules pulls, and normalizes incoming records into a unified schema. This is where the platform's "plumbing" replaces what would otherwise be manual spreadsheet reconciliation: sales, refunds, inventory, expenses, and ad spend become comparable across systems because they share canonical fields (timestamps, product IDs/SKUs, customer identifiers, location, channel). Below is an example target system scenario used to illustrate what Gemini CLI may help build; it is not part of Gemini CLI's internal architecture.

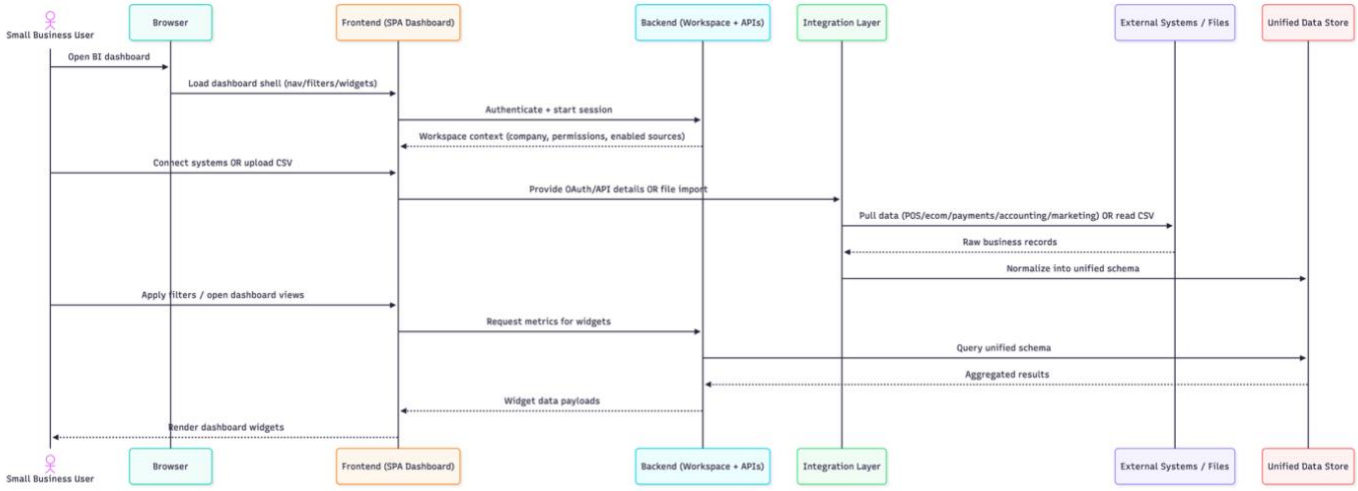

Figure 5: Sequence Diagram of Business Intelligence Dashboard Use Case

6 Lessons Learned

One of the main challenges was determining the correct level of granularity for the conceptual architecture. Early versions of the box-and-arrow diagram either hid responsibilities or became too implementation-level. Maintaining approximately ten subsystems provided a balance between clarity and accuracy and prevented the architecture from turning into a code diagram.

We learned that repository structure does not directly equal architectural structure. Although package boundaries helped identify candidates for subsystems, responsibilities such as prompting, tool execution, and API communication had to be grouped conceptually rather than copied from the codebase.

Creating the sequence diagram repeatedly exposed architectural mistakes. Whenever interactions did not align with the box-and-arrow diagram, it indicated missing components or

misplaced responsibilities. The sequence diagram therefore acted as a validation mechanism for the conceptual architecture rather than just documentation.

The tool-confirmation step initially appeared to be a user-interface detail, but analysis revealed it was part of the system's safety contract. This interaction belongs at the boundary between the user and the system and influences architectural responsibility assignment.

We found that documentation sources describe the same feature at different abstraction levels. Terms such as tools, API, and local system required reconciliation into a single consistent conceptual model, reinforcing that conceptual architecture requires interpretation rather than transcription.

Clear legends and naming conventions significantly improved diagram readability. A consistent representation of synchronous calls, asynchronous calls, and tool invocations allowed the diagrams to be understood without verbal explanation.

Finally, defining concrete use cases early would have simplified the modeling process. Selecting scenarios such as fixing lint errors clarified which components were necessary and helped determine correct interaction paths.

5 Conclusion

Gemini CLI is a great example of a modern layered and interpreter based architectural style designed for AI-assisted software. Through our analysis of its monorepo structure, orchestration flow, and tool invocation cycle, it became clear that that the system is divided between a user-facing CLI layer and a backend core orchestrator that is responsible for managing the agentic loop. This separation of concerns enhances modularity and maintainability, while the interpreter style loop, in which the model determines whether to respond directly or invoke other tools, positions Gemini CLI as a hybrid between a traditional command-line utility and an AI driven automation engine.

Architecturally, Gemini CLI demonstrates several strengths. It's layered design cleanly isolates presentation, orchestration, tool execution, and external API communication. The tool registry used enables extensibility, allowing new capabilities to be added without damaging the system. Additionally, by delegating specialized functionality to external tools, the system is more capable of following the Unix philosophy of composition over reinvention. However, this architecture has its flaws. By relying heavily on external API's, there is an increase in dependency complexity, and the agentic loop can introduce unpredictability compared to deterministic pipelines. These tradeoffs reflect broader tensions in AI-integrated systems between autonomy, control, and reliability.

In the broader context of software architecture, Gemini CLi represents an emerging class of AI developer tools. Unlike traditional CLI applications that execute predefined commands, Gemini CLI interprets high-level intent and constructs execution plans. This shift highlights a movement away from strictly procedural automation and towards adaptive orchestration systems that blend human input, local system resources and cloud-based intelligence.

Ultimately, Gemini CLI illustrates how classical architectural principles, such as layering, separation of concerns, modularity and well-defined interfaces, remain fundamental in new age AI driven systems. While the power of AI is a driving factor in large language models, the key to its success is found in the deliberate architectural styles that structure, constrain, and orchestrate the success of the AI. As AI continues to evolve and integrate more thoroughly into development workflows, systems like Gemini CLI demonstrate that strong architectural designs will remain essential for the success of these LLM's.

7 Data Dictionary

Gemini CLI: Gemini CLI is an open-source command line interface designed by Google to integrate the Gemini Large Language Models into developer terminals, enabling AI assisted automation.

Core Orchestrator: The central coordinating component that manages the agentic loop, routes requests to tools or APIs, and controls the overall execution flow.

Agentic Loop: The processing loop within Gemini CLI which decides user intent, chooses to invoke tools or respond directly, and executes plans until the output is produced.

Tool Registry: The subsystem that maintains the set of tools available to Gemini CLI, for example: File access, or shell command tools.

Interpreter Architecture: An architectural style in which a system repeatedly interprets input and determines the next action to execute, rather than following a predefined execution script.

Layered Architecture: An architectural style in which system responsibilities are organized into layers with well-defined boundaries, such as user interaction, orchestration, tool execution, and external communication.

8 Naming Conventions

CLI (Command Line Interface): A text-based user interface through which users interact with Gemini CLI by issuing commands in a terminal environment.

LLM (Large Language Model): A machine learning model capable of interpreting natural language prompts and generating code, analysis, or automation steps.

API (Application Programming Interface): A network-accessible interface used to communicate with external services that allows different pieces of software to interact and communicate with each other.

9 AI Collaboration Report

a. Task Assignment and Division of Responsibilities

We maintained a clear division of responsibilities between human team members and the AI assistant. AI outputs were always treated as draft-level suggestions and required human

validation before inclusion in the report. This structured division ensured that the architectural conclusions remained human-derived while the AI functioned as an analytical and editorial support tool.

b. AI Interaction Protocol and Prompting Strategy

Our interaction with ChatGPT followed a consistent and reproducible process. Before asking conceptual questions, we provided structured context such as excerpts from documentation or descriptions of Gemini CLI components. Prompts were specific and task-oriented, with examples including "Does this agentic loop align with Interpreter architecture as defined by Garlan & Shaw?", "Clarify the difference between orchestration logic and tool registry responsibilities," and "Refine this explanation to better align with architectural terminology." AI responses were evaluated against official documentation. If inconsistencies were detected, follow-up prompts were issued to refine or challenge the explanation. No architectural classification was accepted solely based on AI suggestion. All architectural claims were cross-checked against source documentation. After human drafting, AI was used to improve clarity, reduce redundancy, and strengthen technical precision without altering meaning. This interaction protocol ensured reproducibility: another group could replicate our process by using structured prompts, documentation-based validation, and iterative refinement.

c. Reflection on Human-AI Team Dynamics

ChatGPT functioned as a cognitive amplifier rather than a replacement contributor. It was particularly useful in clarifying ambiguous architectural distinctions, reframing explanations in more formal academic language, and surfacing potential inconsistencies in subsystem responsibility descriptions. However, AI occasionally produced explanations that were overly generic or slightly misaligned with the specific Gemini CLI implementation. This reinforced the importance of human domain judgment and documentation-based verification. The most productive collaboration occurred when AI was given structured, precise prompts. Vague prompts resulted in high-level answers, whereas targeted prompts yielded actionable clarifications. The AI was most impactful during refinement stages rather than during initial architectural discovery. Core insights, such as identifying the interpreter-style agentic loop and layered subsystem separation, originated from human analysis of documentation.

d. Quantitative Impact

We estimate that AI assistance reduced overall workload by approximately 30%. This impact came primarily from accelerating clarification of architectural terminology, reducing drafting and editing time, assisting in refining diagram explanations, and streamlining technical phrasing and cohesion. Importantly, AI did not generate the architectural conclusions or diagrams independently. Its impact was concentrated in explanation refinement and conceptual validation rather than architectural discovery.

e. Overall Assessment

Our collaboration with ChatGPT was structured, iterative, and verification-driven. The AI enhanced clarity and efficiency while human team members retained responsibility for architectural interpretation, validation, and final decision-making. This structured human-AI workflow reflects the same architectural principles discussed in our report: clear boundaries, defined responsibilities, and controlled orchestration lead to more reliable and maintainable systems.

Works Cited

1. google-gemini/gemini-cli [Internet]. google-gemini; 2026 [cited 2026 Feb 13]. Available from: https://github.com/google-gemini/gemini-cli

2. Gemini CLI [Internet]. [cited 2026 Feb 13]. Gemini CLI documentation. Available from: https://geminicli.com/docs/

3. Garlan D, Shaw M. An Introduction to Software Architecture. Vol. I. World Scientific;